Laurea Triennale in informatica-Università di Salerno Corso di Ingegneria del Software- Prof.ssa F. Ferrucci, Prof. F. Palomba

Test Case Specification

C-09 SINTONIA

Riferimento	2025_C09_TCS
Versione	1.0
Data	06/12/2025
Destinatario	Prof.ssa Filomena Ferrucci, Prof.re Fabio Palomba
Presentato da	C09 Team Sintonia
Approvato da	//

Revision History

Data	Versione	Descrizione	Autori
06/12/2025	1.0	Configurazione Documento	Mattia Fanzini
06/12/2025	1.0	Aggiunti Test Cases	Gavino De Stefano
08/12/2025	1.0	Rivista formattazione e stile	Francesco Coppola

Team Members

Ruolo	Nome e Cognome	Acronimo	Contatto
PM	Gianfranco Barba	GB	g.barba14@studenti.unisa.it
PM	Francesco Corcione	FC	f.corcione5@studenti.unisa.it
TM	Gianluca Ambrosio	GA	g.ambrosio39@studenti.unisa.it
TM	Elena Carlomagno	EC	e.carlomagno1@studenti.unisa.it
TM	Rosaria Cervino	RC	r.cervino3@studenti.unisa.it
TM	Francesco Coppola	FCP	f.coppola99@studenti.unisa.it
TM	Antonio Walter De Fusco	ADF	a.defusco1@studenti.unisa.it
TM	Gavino De Stefano	GDS	g.destefano41@studenti.unisa.it
TM	Alessio Del Sorbo	ADS	a.delsorbo32@studenti.unisa.it
TM	Francesco Di Giovanni	FDG	f.digiovanni10@studenti.unisa.it
TM	Mattia Fanzini	MF	m.fanzini@studenti.unisa.it
TM	Angelo Fusco	AF	a.fusco58@studenti.unisa.it
TM	Gianni Policola	GP	g.policola@studenti.unisa.it

Sommario

Revision History **2**
Team Members 3
Sommario **4**
1. Test case di unità **6**
1.1.1 Compilazione Questionario (UC_PZT_4) 6
1.1.1.1 Unit 1: Validazione Compilazione Questionario (Service) 6
1.1.1.2 Unit 2: Esecuzione Recupero Domande (Service) 9
1.1.1.3 Unit 3: Validazione Calcolo Score (Service) 13
1.1.1.4 Unit 4: Esecuzione Calcolo Score (Service) 17
1.1.1.5 Unit 5: Validazione Invio (Service) 19
1.1.1.6 Unit 6: Esecuzione Invio (Service) 23
1.1.1.7 Unit 7: Gestione Richiesta Get (Controller) 25
1.1.1.8 Unit 8: Gestione Richiesta Start (Controller) 27
1.1.2 Inserimento Stato D'Animo (UC_PZT_5) 31
1.1.2.1 Unit 1 Validazione dati (Service) 31
1.1.2.2 Unit 2: Esecuzione creazione (Service) 40
1.1.2.3 Unit 3: Gestione Richiesta (Controller) 43
1.1.3 Inserimento Contenuto Diario Personale (UC_PZT_19) 46
1.1.3.1 Unit 1: Validazione Contenuto Diario (service) 46
1.1.3.2 Unit 2: Creazione Pagina Diario (Service) 50
1.1.3.3 Unit 3: Creazione Pagina Diario (Controller) 53
1.1.4 Inserimento Domanda Forum (UC_PZT_23) 57
1.1.4.1 Unit 1: Validazione Domanda (service) 57
1.1.4.2 Unit 2: Inserimento Domanda (service) 63
1.1.4.3 Unit 3: Inserimento Domanda (Control) 66
1.2 Test Cases Psicologo 70
1.2.1 Revisione Questionario Paziente (UC_PSI_1) 70
1.2.1.1 Unit 1: Validazione Revisione (Service) 70
1.2.1.2 Unit 2: Esecuzione Revisione Questionario (Service) 77
1.2.1.3 Unit 3: Gestione Richiesta (Controller) 79
1.2.2 Visualizzazione Alert Clinico (UC_PSI_3) 82
1.2.2.1 Unit 1: Esecuzione Visualizzazione Alert (Service) 82
1.2.2.2 Unit 2: Gestione visualizzazione (Controller) 84
1.2.3 Termina Percorso Paziente (UC_PSI_7) 87
1.2.3.1 Unit 1: Validazione Terminazione Cura (Service) 87
1.2.3.2 Unit 2: Esecuzione Terminazione Cura (Service) 93
1.2.3.3 Unit 3: Gestione Richiesta (Controller) 96

1.2.4 Generazione Report (UC_PSI_5) 99
1.2.4.1 Unit 1: Validazione Generazione Report (Service) 99
1.2.4.2 Unit 2: Esecuzione Generazione Report (Service) 102
1.2.4.3 Unit 3: Gestione Richiesta (Controller) 105
1.3 Test Cases Amministratore 108
1.3.1 Modifica Assegnazione Psicologo (UC_AS_11) 108
1.3.1.1 Unit 1: Validazione Assegnazione (Service) 108
1.3.1.2 Unit 2: Esecuzione Assegnazione (Service) 113
1.3.1.3 Unit 3: Gestione Richiesta (Controller) 117
1.3.2 Modifica Priorità Paziente (UC_AS_10) 120
1.3.2.1 Unit 1: Validazione Modifica Priorità (Service) 120
1.3.2.2 Unit 2: Esecuzione Modifica Priorità (Service) 125
1.3.2.3 Unit 3: Gestione Richiesta (Controller) 127
1.3.3 Accettazione Richiesta Invalidazione (UC_AS_17) 130
1.3.3.1 Unit 1: Validazione richiesta (Service) 130
1.3.3.2 Unit 2: Ricerca precedente questionario con cambiamento (service) 136
1.3.3.3 Unit 3: Ricalcolo score (service) 142
1.3.3.4 Unit 4: Downgrade priorità (service) 149
1.3.3.5 Unit 5: Esecuzione Invalidazione (service) 155
1.3.3.6 Unit 6: Gestione Richiesta (Controller) 157

1. Test case di unità

1.1 Test cases paziente

1.1.1 Compilazione Questionario (UC_PZT_4)

1.1.1.1 Unit 1: Validazione Compilazione Questionario (Service)

Test case ID	**TC_RF4_1**	**Test frame**	**VALQ1**
Pre-condizione			
Il Service è inizializzato. ID Questionario vuoto.			
Flusso di eventi			
1. Viene invocato il metodo valdazioneRecupero() del service.			
Input		**Valore**	
ID Questionario		“” (Stringa Vuota)	
Oracolo			
L’operazione fallisce. Il metodo lancia un’eccezione BadRequestException con messaggio “ID Questionario o Nome Tipologia obbligatorio”.			

Test case ID	TC_RF4_2	Test frame	VALQ2, VALT1
Pre-condizione			
Il Service è inizializzato. Input non è UUID e Tipologia non trovata nel DB.			
Flusso di eventi			
1. Viene invocato il metodo valdazioneRecupero(). 2. Controllo formato UUID fallisce. 3. Query DB Tipologia ritorna vuoto.			
Input	**Valore**		
ID Questionario	"NonEsistente"		
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione NotFoundException con messaggio "Tipologia questionario non trovata".			

Test case ID	TC_RF4_3	Test frame	VALQ2, VALT2
Pre-condizione			
Il Service è inizializzato. Input è Tipologia valida.			
Flusso di eventi			
1. Viene invocato validazioneRecupero(). 2. Controllo formato UUID fallisce. 3. Query DB Tipologia ritorna risultato.			
Input	**Valore**		
ID Questionario	"TipologiaEsistente"		
Oracolo			
La validazione termina con successo e restituisce true.			

1.1.1.2 Unit 2: Esecuzione Recupero Domande (Service)

Test case ID	TC_RF4_4	Test frame	EUUID1
Pre-condizione			
Il Service è inizializzato. UUID valido ma non presente in DB.			
Flusso di eventi			
1. Viene invocato getQuestionarioDto(). 2. Query DB Questionario ritorna vuoto.			
Input	**Valore**		
ID Questionario	"00000000000-00000-00000-0000000000"		
Oracolo			
L'operazione fallisce. Il metodo lancia eccezione con messaggio "Questionario non trovato".			

Test case ID	TC_RF4_5	Test frame	ETIP1
Pre-condizione			
Il Service è inizializzato. Input non UUID, Tipologia non trovata.			
Flusso di eventi			
1. Viene invocato getQuestionarioDto(). 2. Query DB Tipologia ritorna vuoto.			
Input		**Valore**	
ID Questionario		"TipoErrato"	
Oracolo			
L'operazione fallisce. Il metodo lancia eccezione con messaggio "Tipologia non trovata".			

Test case ID	TC_RF4_6	Test frame	EUUID2, ETIP1
Pre-condizione			
Il Service è inizializzato. UUID trovato ma Tipologia associata inconsistente (non trovata).			
Flusso di eventi			
1. Viene invocato getQuestionarioDto(). 2. Query Questionario OK. 3. Query Tipologia fallisce.			
Input	**Valore**		
ID Questionario	“1111111111111111-1111111-111111-111111”		
Oracolo			
L’operazione fallisce. Il metodo lancia eccezione con messaggio “Tipologia non trovata”.			

Test case ID	TC_RF4_7	Test frame	EUUID2, ETIP2
Pre-condizione			
Il Service è inizializzato. Dati coerenti.			
Flusso di eventi			
1. Viene invocato getQuestionarioDto(). 2. Tutte le query hanno successo. 3. Parsing domande eseguito.			
Input	**Valore**		
ID Questionario	"2222222-22222-22222-22222222"		
Oracolo			
L'operazione termina con successo. Restituisce il DTO del questionario popolato.			

1.1.1.3 Unit 3: Validazione Calcolo Score (Service)

Test case ID	TC_RF4_8	Test frame	VALSC1
Pre-condizione			
Il Service è inizializzato. Nome Tipologia mancante.			
Flusso di eventi			
1. Viene invocato validazioneCalcoloScore().			
Input	**Valore**		
Nome Tipologia	“”		
Risposte	[]		
Oracolo			
L’operazione fallisce. BadRequestException con messaggio “Nome tipologia obbligatorio”.			

Test case ID	TC_RF4_9	Test frame	VALSC2
Pre-condizione			
Il Service è inizializzato. Tipologia non trovata nel DB.			
Flusso di eventi			
1. Viene invocato validazioneCalcoloScore(). 2. Query DB Tipologia fallisce.			
Input	**Valore**		
Nome Tipologia	"NonEsistente"		
Risposte	[]		
Oracolo			
L'operazione fallisce. NotFoundException con messaggio "Tipologia questionario non trovata".			

Test case ID	TC_RF4_10	Test frame	VALSC3, VALR1
Pre-condizione			
Il Service è inizializzato. Risposte non valide (null o non array).			
Flusso di eventi			
1. Viene invocato validazioneCalcoloScore(). 2. Checks tipologia superati. 3. Check array fallisce.			
Input	**Valore**		
Nome Tipologia	"Esistente"		
Risposte	null		
Oracolo			
L'operazione fallisce. BadRequestException con messaggio "Le risposte devono essere un array valido".			

Test case ID	TC_RF4_11	Test frame	VALSC3, VALR2
Pre-condizione			
Il Service è inizializzato. Input corretti.			
Flusso di eventi			
1. Viene invocato validazioneCalcoloScore(). 2. Tutti i controlli passano.			
Input	**Valore**		
Nome Tipologia	“Esistente”		
Risposte	[{ idDomanda: “q1”, valore: 1 }]		
Oracolo			
La validazione termina con successo e restituisce true.			

1.1.1.4 Unit 4: Esecuzione Calcolo Score (Service)

Test case ID	TC_RF4_12	Test frame	VALR1
Pre-condizione			
Il Service è inizializzato. Risposte vuote o invalide per il calcolo.			
Flusso di eventi			
1. Viene invocato calculateScore(). 2. Logica punteggio non trova corrispondenze valide.			
Input	**Valore**		
Nome Tipologia	"Test"		
Risposte	[]		
Oracolo			
L'operazione termina con successo, ma ritorna Score 0.			

Test case ID	TC_RF4_13	Test frame	VALR2
Pre-condizione			
Il Service è inizializzato. Risposte valide e config presente.			
Flusso di eventi			
1. Viene invocato calculateScore(). 2. Logica punteggio non trova corrispondenze valide.			
Input	**Valore**		
Nome Tipologia	"Test"		
Risposte	[{ idDomanda: "q1", valore: 2 }]		
Oracolo			
L'operazione termina con successo. Ritorna Score calcolato (es. 100).			

1.1.1.5 Unit 5: Validazione Invio (Service)

Test case ID	TC_RF4_14	Test frame	VALPID1
Pre-condizione			
Il Service è inizializzato. ID Paziente mancante.			
Flusso di eventi			
1. Viene invocato validazioneInvio().			
Input		**Valore**	
ID Paziente		“”	
Oracolo			
Fallisce con BadRequestException (“ID Paziente obbligatorio”).			

Test case ID	TC_RF4_15	Test frame	VALPID2
Pre-condizione			
Il Service è inizializzato. Paziente non trovato.			
Flusso di eventi			
1. Viene invocato validazioneInvio(). 2. Query Paziente fallisce.			
Input		**Valore**	
ID Paziente		“missing-patient”	
Oracolo			
Fallisce con NotFoundException (“Paziente non trovato”).			

Test case ID	TC_RF4_16	Test frame	VALPID3, VALRIS1
Pre-condizione			
Il Service è inizializzato. Paziente OK, ma risposte vuote.			
Flusso di eventi			
1. Viene invocato validazioneInvio(). 2. Check array risposte fallisce.			
Input	**Valore**		
ID Paziente	“existing-patient”		
Risposte	[]		
Oracolo			
Fallisce con BadRequestException (“è necessario fornire almeno una risposta”).			

Test case ID	TC_RF4_17	Test frame	VALPID3, VALRIS2
Pre-condizione			
Il Service è inizializzato. Dati validi.			
Flusso di eventi			
1. Viene invocato validazioneInvio(). 2. Tutti i check passano.			
Input	**Valore**		
ID Paziente	“existing-patient”		
Risposte	[{ … }]		
Oracolo			
Successo, ritorna true.			

1.1.1.6 Unit 6: Esecuzione Invio (Service)

Test case ID	**TC_RF4_18**	**Test frame**	**DB1**
Pre-condizione			
Il Service è inizializzato. Dati validi.			
Flusso di eventi			
1. Viene invocato submitQuestionario(). 2. db.insert lancia errore.			
Input	**Valore**		
Dati	Validi		
Oracolo			
Fallisce con eccezione “Impossibile salvare il questionario”.			

Test case ID	TC_RF4_19	Test frame	DB2
Pre-condizione			
Il Service è inizializzato. Insert OK.			
Flusso di eventi			
1. Viene invocato submitQuestionario(). 2. db.insert successo. 3. Update score/alert/badge eseguiti.			
Input	**Valore**		
Dati	Validi		
Oracolo			
Successo. Ritorna oggetto { idQuestionario, score}.			

1.1.1.7 Unit 7: Gestione Richiesta Get (Controller)

Test case ID	**TC_RF4_20**	**Test frame**	**GET1**
Pre-condizione			
Controller pronto. Service lancia errore.			
Flusso di eventi			
1. getQuestionario() chiama service. 2. Service fallisce.			
Input		**Valore**	
Params		{ id: “invalid” }	
Oracolo			
Controller propaga l’eccezione.			

Test case ID	TC_RF4_21	Test frame	GET2
Pre-condizione			
Controller pronto. Service OK.			
Flusso di eventi			
1. getQuestionario() chiama service. 2. Service ritorna DTO.			
Input		**Valore**	
Params		{ id: “invalid” }	
Oracolo			
Ritorna 200 OK con DTO.			

1.1.1.8 Unit 8: Gestione Richiesta Start (Controller)

Test case ID	**TC_RF4_22**	**Test frame**	**START1**
Pre-condizione			
Controller pronto. Service lancia errore (es.Tipologia errata).			
Flusso di eventi			
1. startCompilazione() chiama service.			
Input		**Valore**	
Body		{ nomeTipologia: “Invalid” }	
Oracolo			
Controller propaga eccezione.			

Test case ID	TC_RF4_23	Test frame	START2
Pre-condizione			
Controller pronto. Service OK.			
Flusso di eventi			
1. startCompilazione() chiama service.			
Input		**Valore**	
Body		{ nomeTipologia: “Valid” }	
Oracolo			
Ritorna 201 Created/200 OK con dati questionario.			

Test case ID	TC_RF4_24	Test frame	SUB1
Pre-condizione			
Controller pronto. Service fallisce (validazione).			
Flusso di eventi			
1. submitQuestionario() chiama service.			
Input	**Valore**		
Body	Invalido		
Oracolo			
Controller propaga eccezione.			

Test case ID	TC_RF4_25	Test frame	SUB2
Pre-condizione			
Controller pronto. Service OK.			
Flusso di eventi			
1. submitQuestionario() chiama service.			
Input		**Valore**	
Body		Valido	
Oracolo			
Ritorna 201 Created con Risultato e Score.			

1.1.2 Inserimento Stato D'Animo (UC_PZT_5)

1.1.2.1 Unit 1 Validazione dati (Service)

Test case ID	**TC_RF5_1**	**Test frame**	**VALPAZ1**
Pre-condizione			
Il Service è inizializzato. Data input non valida.			
Flusso di eventi			
4. Viene invocato il metodo validazione() del service.			
Input	**Valore**		
Patient ID	" " (stringa vuota)		
DTO	{ umore: "Felice", intensità: 5}		
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione BadRequestException con messaggio "ID paziente obbligatorio".			

Test case ID	TC_RF5_2	Test frame	VALPAZ2
Pre-condizione			
Il Service è inizializzato. Paziente non trovato nel DB.			
Flusso di eventi			
1. Viene invocato validazione(). Query DB ritorna vuoto			
Input	**Valore**		
Patient ID	"paziente-inesistente"		
DTO	{ umore: "Felice", intensità: 5}		
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione NotFoundException con messaggio "paziente con ID … non trovato".			

Test case ID	TC_RF5_3	Test frame	VALPAZ3, VALUM1
Pre-condizione			
Il Service è inizializzato. Paziente esiste, Umore mancante.			
Flusso di eventi			
1. Viene invocato validazione().			
Input	**Valore**		
Patient ID	“paziente-esistente”		
DTO	{ umore: “null”, intensità: 5}		
Oracolo			
L’operazione fallisce. Il metodo lancia un eccezione BadRequestException con messaggio “L’umore è obbligatorio”.			

Test case ID	TC_RF5_4	Test frame	VALPAZ3, VALUM2
Pre-condizione			
Il Service è inizializzato. Umore tipo errato.			
Flusso di eventi			
1. Viene invocato validazione().			
Input	**Valore**		
Patient ID	“paziente-esistente”		
DTO	{ umore: 123, intensità: 5}		
Oracolo			
L’operazione fallisce. Il metodo lancia un’eccezione BadRequestException con messaggio “L’umore è obbligatorio” (check tipo stringa).			

Test case ID	TC_RF5_5	Test frame	VALPAZ3, VALUM3
Pre-condizione			
Il Service è inizializzato. Umore non in Enum.			
Flusso di eventi			
1. Viene invocato validazione().			
Input	**Valore**		
Patient ID	“paziente-esistente”		
DTO	{ umore: “Inesistente”, intensità: 5}		
Oracolo			
L’operazione fallisce. Il metodo lancia un’eccezione BadRequestException con messaggio “L’umore deve essere uno dei seguenti valori…”.			

Test case ID	TC_RF5_6	Test frame	VALPAZ3, VALINT3
Pre-condizione			
Il Service è inizializzato. Intensità fuori range.			
Flusso di eventi			
1. Viene invocato validazione().			
Input	**Valore**		
Patient ID	“paziente-esistente”		
DTO	{ umore: “Felice”, intensità: 11}		
Oracolo			
L’operazione fallisce. Il metodo lancia un’eccezione BadRequestException con messaggio “L’intensità deve essere compresa tra 1 e 10”.			

Test case ID	TC_RF5_7	Test frame	VALPAZ3, VALNOTE2
Pre-condizione			
Il Service è inizializzato. Note troppo lunghe.			
Flusso di eventi			
1. Viene invocato validazione().			
Input	**Valore**		
Patient ID	“paziente-esistente”		
DTO	{ umore: “Felice”, intensità: 5, note: “LONG…” }		
Oracolo			
L’operazione fallisce. Il metodo lancia un’eccezione BadRequestException con messaggio “Le note non possono superare i 500 caratteri”.			

Test case ID	TC_RF5_8	Test frame	VALPAZ3, VALDATA2
Pre-condizione			
Il Service è inizializzato. Data non valida.			
Flusso di eventi			
1. Viene invocato validazione().			
Input	**Valore**		
Patient ID	“paziente-esistente”		
DTO	{ umore: “Felice”, intensità: 5, dataInserimento: “err” }		
Oracolo			
L’operazione fallisce. Il metodo lancia un’eccezione BadRequestException con messaggio “La data di inserimento non è valida”.			

Test case ID	TC_RF5_9	Test frame	VALPAZ3, VALUM4, VALINT 4, VALNOTE3, VALDATA3
Pre-condizione			
Il Service è inizializzato. Tutti i dati sono validi.			
Flusso di eventi			
1. Viene invocato validazione().			
Input	**Valore**		
Patient ID	“paziente-esistente”		
DTO	{ umore: “Felice”, intensità: 5, ... }		
Oracolo			
La validazione termina con successo.			

1.1.2.2 Unit 2: Esecuzione creazione (Service)

Test case ID	TC_RF5_10	Test frame	DB1
Pre-condizione			
Il Service è inizializzato. Validazione superata. Errore insert DB.			
Flusso di eventi			
1. Viene invocato createStatoAnimo(). 2. Insert fallisce.			
Input	**Valore**		
Patient ID	"paziente-esistente"		
DTO	{ umore: "Felice", intensità: 8 }		
Oracolo			
L'operazione fallisce. L'eccezione viene propagata (o "Impossibile creare…").			

Test case ID	TC_RF5_11	Test frame	DB2, BADGE1
Pre-condizione			
Il Service è inizializzato. Insert OK. Badge Service Errore.			
Flusso di eventi			
1. Viene invocato createStatoAnimo(). 2. Insert OK. 3. Check Badge fallisce.			
Input	**Valore**		
Patient ID	“paziente-esistente”		
DTO	{ umore: “Felice”, intensità: 8 }		
Oracolo			
L’operazione fallisce. L’eccezione Badge viene propagata.			

Test case ID	TC_RF5_12	Test frame	DB2, BADGE2
Pre-condizione			
Il Service è inizializzato. Flusso success.			
Flusso di eventi			
1. Viene invocato createStatoAnimo().			
Input	**Valore**		
Patient ID	“paziente-esistente”		
DTO	{ umore: “Felice”, intensità: 8 }		
Oracolo			
Operazione riuscita. Ritorna oggetto (id, dataInserimento).			

1.1.2.3 Unit 3: Gestione Richiesta (Controller)

Test case ID	**TC_RF5_13**	**Test frame**	**DLG1**
Pre-condizione			
Il Controller è inizializzato. Parametri errati (es. umore mancante).			
Flusso di eventi			
1. Viene invocato createStatoAnimo() del Controller.			
Input	**Valore**		
Req.User	{ id: “paziente-esistente” }		
Body	{ }		
Oracolo			
Il controller propaga l’eccezione del service (BadRequestException).			

Test case ID	TC_RF5_14	Test frame	DLG2, RSP1
Pre-condizione			
Il Controller è inizializzato. Service fallisce (es. DB Error).			
Flusso di eventi			
1. Controller invoca Service. 2. Service lancia eccezione.			
Input	**Valore**		
Req.User	{ id: “paziente-esistente” }		
Body	{umore: “Felice”, intensita: 8 }		
Oracolo			
Il controller propaga l’eccezione.			

Test case ID	TC_RF5_15	Test frame	DLG2, RSP2
Pre-condizione			
Il Controller è inizializzato. Service OK.			
Flusso di eventi			
1. Controller invoca Service. 2. Service ritorna dati.			
Input		**Valore**	
Req.User		{ id: “paziente-esistente” }	
Body		{umore: “Felice”, intensita: 8 }	
Oracolo			
Il controller restituisce 201 Created (implicito) e il payload.			

1.1.3 Inserimento Contenuto Diario Personale (UC_PZT_19)

1.1.3.1 Unit 1: Validazione Contenuto Diario (service)

Test case ID	**TC_RF19_1**	**Test frame**	**LTD1**
Pre-condizione			
Il Service è inizializzato correttamente.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service con il seguente oggetto DTO in input.			
Input	**Valore**		
Titolo	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA (65 chars)		
Contenuto	Contenuto valido		
Oracolo			
L'inserimento non va a buon fine. Il metodo lancia un'eccezione BadRequestException con messaggio "Il titolo non può superare i 64 caratteri".			

Test case ID	TC_RF19_2	Test frame	LTD2, LCD1
Pre-condizione			
Il DiarioService è inizializzato correttamente.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service con il seguente oggetto DTO in input.			
Input	**Valore**		
Titolo	Titolo valido		
Contenuto	“” (Stringa vuota)		
Oracolo			
L'inserimento non va a buon fine. Il metodo lancia un'eccezione BadRequestException con messaggio "Il contenuto è obbligatorio".			

Test case ID	TC_RF19_3	Test frame	LTD2, LCD2
Pre-condizione			
Il Service è inizializzato correttamente.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service con il seguente oggetto DTO in input.			
Input	**Valore**		
Titolo	Titolo valido		
Contenuto	(Stringa > 2000 caratteri)		
Oracolo			
L'inserimento non va a buon fine. Il metodo lancia un'eccezione BadRequestException con messaggio "Il contenuto non può superare i 2000 caratteri".			

Test case ID	TC_RF19_4	Test frame	LTD2, LCD3
Pre-condizione			
Il Service è inizializzato correttamente.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service con il seguente oggetto DTO in input.			
Input	**Valore**		
Titolo	Titolo valido		
Contenuto	Contenuto valido (es. “Oggi è una bella giornata...”)		
Oracolo			
La validazione va a buon fine. Il metodo non lancia eccezioni e l'esecuzione prosegue.			

1.1.3.2 Unit 2: Creazione Pagina Diario (Service)

Test case ID	TC_RF19_5	Test frame	DB1
Pre-condizione			
Il Service è inizializzato. Il Database simula un errore durante l'inserimento (o restituisce null).			
Flusso di eventi			
1. Viene invocato il metodo createDiaryPage(). 2. Il DB fallisce l'operazione.			
Input	**Valore**		
ID Paziente	paziente-123		
DTO	{ Titolo: "Valido", Contenuto: "Valido" }		
Oracolo			
L'operazione fallisce. Il metodo lancia BadRequestException con messaggio "Impossibile creare la pagina del diario".			

Test case ID	TC_RF19_6	Test frame	DB2, BDG1
Pre-condizione			
Il Service è inizializzato. Database OK. Service Badge non viene chiamato (Simulazione errore logico).			
Flusso di eventi			
1. Viene invocato createDiaryPage(). 2. DB inserisce con successo. 3. BadgeService NON viene invocato.			
Input	**Valore**		
ID Paziente	paziente-123		
DTO	{ Titolo: "Valido", Contenuto: "Valido" }		
Oracolo			
(Test Negativo Simulato) Il test framework rileva che badgeService.checkAndAwardBadges non è stato chiamato, fallendo l'asserzione.			

Test case ID	TC_RF19_7	Test frame	DB2, BDG2
Pre-condizione			
Il Service è inizializzato. Database e BadgeService funzionano correttamente (Mock).			
Flusso di eventi			
1. Viene invocato il metodo createDiaryPage(). 2. DB inserisce la pagina. 3. BadgeService assegna badge.			
Input	**Valore**		
ID Paziente	paziente-123		
DTO	{ Titolo: "Valido", Contenuto: "Valido" }		
Oracolo			
L'operazione termina con successo. Viene restituito l'oggetto DTO della pagina creata e il BadgeService viene invocato.			

1.1.3.3 Unit 3: Creazione Pagina Diario (Controller)

Test case ID	TC_RF19_8	Test frame	VALID1
Pre-condizione			
Il Controller è inizializzato. La Request HTTP non contiene l'oggetto user o id.			
Flusso di eventi			
1. Viene invocato il metodo createDiaryPage() del controller.			
Input	**Valore**		
Request	{ user: {} } (ID Mancante)		
DTO	{ Titolo: "Ok", Contenuto: "Ok" }		
Oracolo			
Il controller lancia un'eccezione Error con messaggio "User ID non trovato".			

Test case ID	TC_RF19_9	Test frame	VALID2, DLG1
Pre-condizione			
Il Service è configurato per lanciare un'eccezione (es. validazione fallita).			
Flusso di eventi			
1. Il controller invoca il service. 2. Il service lancia eccezione.			
Input	**Valore**		
Request	{ user: { id: "paziente-123" } }		
DTO	(DTO Invalido)		
Oracolo			
Il controller cattura e rilancia (o lascia propagare) l'eccezione del service: BadRequestException.			

Test case ID	TC_RF19_10	Test frame	VALID2, DLG2, RSP1
Pre-condizione			
Il Service restituisce null o undefined (errore inatteso).			
Flusso di eventi			
1. Il controller invoca il service. 2. Il service non ritorna dati.			
Input	**Valore**		
Request	{ user: { id: "paziente-123" } }		
DTO	{ Titolo: "Ok", Contenuto: "Ok" }		
Oracolo			
Il controller rileva il risultato vuoto e lancia un'eccezione Error con messaggio “Nessun risultato dal service”.			

Test case ID	TC_RF19_11	Test frame	VALID2, DLG2, RSP2
Pre-condizione			
Service Mockato per restituire successo.			
Flusso di eventi			
1. Il controller estrae l'ID dalla request. 2. Invoca il service. 3. Restituisce il risultato.			
Input	**Valore**		
Request	{ user: { id: "paziente-123" } }		
DTO	{ Titolo: "Titolo", Contenuto: "Contenuto" }		
Oracolo			
Il controller restituisce l'oggetto JSON della pagina creata: { id: "...", title: "Titolo", ... }.			

1.1.4 Inserimento Domanda Forum (UC_PZT_23)

1.1.4.1 Unit 1: Validazione Domanda (service)

Test case ID	**TC_RF23_1**	**Test frame**	**LTI1**
Precondizione			
Il ForumService è inizializzato correttamente.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service: inserimento_forum con il seguente oggetto DTO in input:			
Input	**Valore**		
Titolo			
Testo	Testo valido		
Categoria	Vita di Coppia		
Oracolo			
L'inserimento non va a buon fine. Il metodo lancia un'eccezione BadRequestException perché il titolo è vuoto.			

Test case ID	TC_RF23_2	Test frame	LTI2
Pre-condizione			
Il ForumService è inizializzato correttamente.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service con il seguente oggetto DTO in input:			
Input	**Valore**		
Titolo	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA (65 chars)		
Testo	Testo valido.		
Categoria	Vita di Coppia		
Oracolo			
L'inserimento non va a buon fine. Il metodo lancia un'eccezione BadRequestException perché il titolo supera i 64 caratteri.			

Test case ID	TC_RF23_3	Test frame	LTI3, LTE1
Pre-condizione			
Il ForumService è inizializzato correttamente.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service con il seguente oggetto DTO in input:			
Input	**Valore**		
Titolo	Titolo valido		
Testo	“” (Stringa vuota).		
Categoria	Vita di Coppia		
Oracolo			
L'inserimento non va a buon fine. Il metodo lancia un'eccezione BadRequestException perché il testo è obbligatorio.			

Test case ID	TC_RF23_4	Test frame	LTI3, LTE2, SC1
Pre-condizione			
Il ForumService è inizializzato correttamente.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service con il seguente oggetto DTO in input:			
Input	**Valore**		
Titolo	Titolo valido		
Testo	Testo valido		
Categoria	“” (Stringa vuota)		
Oracolo			
L'inserimento non va a buon fine. Il metodo lancia un'eccezione BadRequestException perché la categoria è obbligatoria.			

Test case ID	TC_RF23_5	Test frame	LTI3, LTE2, SC2, VAL1
Pre-condizione			
Il ForumService è inizializzato correttamente.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service con il seguente oggetto DTO in input:			
Input	**Valore**		
Titolo	Titolo valido		
Testo	Testo valido		
Categoria	Generale (Non in whitelist).		
Oracolo			
L'inserimento non va a buon fine. Il metodo lancia un'eccezione BadRequestException perché la categoria non è valida.			

Test case ID	TC_RF23_6	Test frame	LTI3, LTE2, SC2, VAL2
Pre-condizione			
Il ForumService è inizializzato correttamente.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service con il seguente oggetto DTO in input:			

Input	Valore
Titolo	Titolo valido
Testo	Testo valido
Categoria	Vita di Coppia

Oracolo
La validazione va a buon fine. Il metodo non lancia eccezioni e l'esecuzione prosegue.

1.1.4.2 Unit 2: Inserimento Domanda (service)

Test case ID	TC_RF23_7	Test frame	DB1
Pre-condizione			
Il Service è inizializzato. Il Database simula un errore durante l'inserimento.			
Flusso di eventi			
1. Viene invocato il metodo inserisciDomanda(). 2. Il DB fallisce l'operazione.			
Input	**Valore**		
ID_Paziente	paziente-123		
DTO	{ Titolo: "Valido", Testo: "Valido", Categoria: "Vita di Coppia" }		
Oracolo			
L'operazione fallisce. Il metodo lancia Error con messaggio "Impossibile inserire la domanda nel forum".			

Test case ID	TC_RF23_8	Test frame	DB2, BDG1
Pre-condizione			
Il Service è inizializzato. Database OK. Service Badge non viene chiamato (Simulazione errore logico).			
Flusso di eventi			
1. Viene invocato inserisciDomanda(). 2. DB inserisce con successo. 3. BadgeService NON viene invocato.			
Input	**Valore**		
ID_Paziente	paziente-123		
DTO	{ Titolo: "Valido", Testo: "Valido", Categoria: "Vita di Coppia" }		
Oracolo			
(Test Negativo Simulato) Il test framework rileva che badgeService.checkAndAwardBadges non è stato chiamato, fallendo l'asserzione.			

Test case ID	TC_RF23_9	Test frame	DB2, BDG2
Pre-condizione			
Il Service è inizializzato. Database e BadgeService funzionano correttamente (Mock).			
Flusso di eventi			
1. Viene invocato inserisciDomanda(). 2. DB inserisce la domanda 3. BadgeService assegna badge.			
Input	**Valore**		
ID_Paziente	paziente-123		
DTO	{ Titolo: "Valido", Testo: "Valido", Categoria: "Vita di Coppia" }		
Oracolo			
L'operazione termina con successo (success: true). Viene restituito un oggetto con messaggio "Domanda pubblicata con successo...".			

1.1.4.3 Unit 3: Inserimento Domanda (Control)

Test case ID	TC_RF23_10	Test frame	VALID1
Pre-condizione			
Il Controller è inizializzato. La Request HTTP non contiene l'oggetto user o id.			
Flusso di eventi			
1. Viene invocato il metodo inserisciDomanda() del controller.			
Input	**Valore**		
Request	{ user: {} } (ID Mancante)		
DTO	{ Titolo: "Ok", Testo: "Ok", Categoria: "Vita di Coppia" }		
Oracolo			
Il controller lancia un'eccezione (Error o BadRequest) con messaggio "User ID non trovato".			

Test case ID	TC_RF23_11	Test frame	VALID2, DLG1
Pre-condizione			
Il Service è configurato per lanciare un'eccezione (es. validazione fallita).			
Flusso di eventi			
1. Il controller invoca il service. 2. Il server lancia un'eccezione.			
Input	**Valore**		
Request	{ user: { id: "paziente-123+" } }		
DTO	(DTO Invalido)		
Oracolo			
Il controller cattura e rilancia (o lascia propagare) l'eccezione del service: BadRequestException: DTO non valido.			

Test case ID	TC_RF23_12	Test frame	VALID2, DLG2, RSP1
Pre-condizione			
Il Service restituisce null o undefined (errore inatteso).			
Flusso di eventi			
1. Il controller invoca il service. 2. Il service non ritorna i dati.			
Input	**Valore**		
Request	{ user: { id: "paziente-123" } }		
DTO	{ Titolo: "Ok", Testo: "Ok", Categoria: "Vita di Coppia" }		
Oracolo			
Il controller rileva il risultato vuoto e lancia un'eccezione InternalServerError o Error con messaggio "Nessun risultato dal service".			

Test case ID	TC_RF23_13	Test frame	VALID2, DLG2, RSP2
Pre-condizione			
Service Mockato per restituire successo.			
Flusso di eventi			
1. Il controller estrae l'ID dalla request. 2. Invoca il service. 3. Restituisce il risultato.			
Input	**Valore**		
Request	{ user: { id: "paziente-123" } }		
DTO	{ Titolo: "Titolo", Testo: "Testo", Categoria: "Vita di Coppia" }		
Oracolo			
Il controller restituisce l'oggetto di successo JSON: { success: true, idDomanda: "...", message: "OK" }.			

1.2 Test Cases Psicologo

1.2.1 Revisione Questionario Paziente (UC_PSI_1)

1.2.1.1 Unit 1: Validazione Revisione (Service)

Test case ID	**TC_RF1_1**	**Test frame**	**VINQ1**
Pre-condizione			
Il Service è inizializzato. ID Questionario vuoto.			
Flusso di eventi			
1. Viene invocato il metodo validazione().			
Input		**Valore**	
ID Questionario		“” (Stringa Vuota)	
ID Psicologo		“CF_VALIDO”	
Oracolo			
L’operazione fallisce. Eccezione: “ID Questionario obbligatorio”.			

Test case ID	TC_RF1_2	Test frame	VINQ2, VINP1
Pre-condizione			
Il Service è inizializzato. ID Psicologo vuoto.			
Flusso di eventi			
1. Viene invocato il metodo validazione().			
Input	**Valore**		
ID Questionario	“valid-id”		
ID Psicologo	“” (Stringa Vuota)		
Oracolo			
L’operazione fallisce. Eccezione: “ID Psicologoobbligatorio”.			

Test case ID	TC_RF1_3	Test frame	VINQ2, VINP2, VEX1
Pre-condizione			
Il Service è inizializzato. Questionario non esistente.			
Flusso di eventi			
1. Query al DB restituisce set vuoto.			
Input	**Valore**		
ID Questionario	“not-found-id”		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L’operazione fallisce. Eccezione: “Questionario … non trovato”.			

Test case ID	TC_RF1_4	Test frame	VINQ2, VINP2, VEX2, VREV1
Pre-condizione			
Questionario esiste ma revisionato = true.			
Flusso di eventi			
1. Check stato revisione fallisce.			
Input	**Valore**		
ID Questionario	“valid-id”		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L’operazione fallisce. Eccezione: “Il questionario è già stato revisionato”.			

Test case ID	TC_RF1_5	Test frame	VINQ2, VINP2, VEX2, VREV2, VINV1
Pre-condizione			
Questionario esiste, revisionato = false, invalidato = true.			
Flusso di eventi			
1. Check stato invalidazione fallisce.			
Input	**Valore**		
ID Questionario	“valid-id”		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L’operazione fallisce. Eccezione: “Questionario … non trovato”.			

Test case ID	TC_RF1_6	Test frame	VINQ2, VINP2, VEX2, VREV2, VINV2
Pre-condizione			
Questionario esiste, idPsicologoRichiedente != null.			
Flusso di eventi			
1. Check richiesta invalidazione fallisce.			
Input	**Valore**		
ID Questionario	"valid-id"		
ID Psicologo	"CF_VALIDO"		
Oracolo			
L'operazione fallisce. Eccezione: "è stata richiesta l'invalidazione per questo questionario".			

Test case ID	TC_RF1_6_OK	Test frame	VINQ2, VINP2, VEX2, VREV2, VINV3
Pre-condizione			
Tutti i vincoli soddisfatti.			
Flusso di eventi			
1. Validazione passa con successo			
Input	**Valore**		
ID Questionario	“valid-id”		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L’operazione termina con successo.			

1.2.1.2 Unit 2: Esecuzione Revisione Questionario (Service)

Test case ID	**TC_RF1_7**	**Test frame**	**DB1**
Pre-condizione			
Validazione OK. Errore DB simulato.			
Flusso di eventi			
1. revisionaQuestionario chiama update. 2. Update fallisce.			
Input		**Valore**	
ID Questionario		"valid-id"	
ID Psicologo		"CF_VALIDO"	
Oracolo			
L'operazione fallisce. Eccezione: "Database Error".			

Test case ID	TC_RF1_8	Test frame	DB2
Pre-condizione			
DB funzionante.			
Flusso di eventi			
1. revisionaQuestionario chiama update. 2. Update successo.			
Input	**Valore**		
ID Questionario	“valid-id”		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L’operazione termina con successo.			

1.2.1.3 Unit 3: Gestione Richiesta (Controller)

Test case ID	**TC_RF1_9**	**Test frame**	**DLG1**
Pre-condizione			
Parametri input errati (es. UUID non valido).			
Flusso di eventi			
1. Controller invocato.			
Input		**Valore**	
Params		{ id: “invalid-uuid” }	
Query		{ cf: “CF_VALIDO” }	
Oracolo			
BadRequestException (Validation failed).			

Test case ID	TC_RF1_10	Test frame	DLG2, RSP1
Pre-condizione			
Service fallisce silenziosamente o restituisce null non gestito (simulato).			
Flusso di eventi			
1. Controller chiama Service. 2. Errore interno.			
Input		**Valore**	
Params		{ id: “valid-uuid” }	
Query		{ cf: “CF_VALIDO” }	
Oracolo			
InternalServerError.			

Test case ID	TC_RF1_11	Test frame	DLG2, RSP2
Pre-condizione			
Flusso felice.			
Flusso di eventi			
1. Controller chiama Service. 2. Ritorna payload successo.			
Input		**Valore**	
Params		{ id: “valid-uuid” }	
Query		{ cf: “CF_VALIDO” }	
Oracolo			
Status 200 OK, JSON { message: “Questionario revisionato con successo” }.			

1.2.2 Visualizzazione Alert Clinico (UC_PSI_3)

1.2.2.1 Unit 1: Esecuzione Visualizzazione Alert (Service)

Test case ID	**TC_RF3_1**	**Test frame**	**DB1**
Pre-condizione			
Service inizializzato. DB offline o query fallita.			
Flusso di eventi			
1. Invocazione getAlertNonAccettati(). 2. DB query lancia eccezione.			
Input		-	
Eccezione propagata.			

Test case ID	TC_RF3_2	Test frame	DB2
Pre-condizione			
Service inizializzato. DB accessibile.			
Flusso di eventi			
1. Invocazione getAlertNonAccettati(). 2. DB query restituisce righe.			
Input		-	
Oracolo			
Lista alert restituita.			

1.2.2.2 Unit 2: Gestione visualizzazione (Controller)

Test case ID	**TC_RF3_3**	**Test frame**	**DLG1**
Pre-condizione			
Parametri errati passati al controller.			
Input		(Simulazione interna)	
Oracolo			
400 Bad Request o Service non chiamato.			

Test case ID	TC_RF3_4	Test frame	DLG2, RSP1
Pre-condizione			
Service lancia eccezione.			
Input		(Request valida)	
Oracolo			
500 Internal Server Error.			

Test case ID	TC_RF3_5	Test frame	DLG2, RSP2
Pre-condizione			
Service restituisce dati.			
Input		(Request valida)	
Oracolo			
200 OK, Body JSON corretto.			

1.2.3 Termina Percorso Paziente (UC_PSI_7)

1.2.3.1 Unit 1: Validazione Terminazione Cura (Service)

Test case ID	**TC_RF7_1**	**Test frame**	**VPZT1**
Pre-condizione			
Il Service è inizializzato. Data input non valida.			
Flusso di eventi			
1. Viene invocato il metodo validazione del service.			
Input	**Valore**		
ID Paziente	“” (Stringa Vuota)		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L’operazione fallisce. Il metodo lancia un’eccezione BadRequestException con messaggio “ID Paziente obbligatorio”.			

Test case ID	TC_RF7_2	Test frame	VPZT2
Pre-condizione			
Il Service è inizializzato. Paziente non trovato nel DB.			
Flusso di eventi			
1. Viene inovcato validazione(). 2. Query DB ritorna vuoto.			
Input	**Valore**		
ID Paziente	“paziente-inesistente”		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione NotFoundException con messaggio "Paziente con ID ... non trovato".			

Test case ID	TC_RF7_3	Test frame	VPZT3, VAUTH1
Pre-condizione			
Il Service è inizializzato. Paziente esiste.			
Flusso di eventi			
1. Viene invocato validazione().			
Input	**Valore**		
ID Paziente	“paziente-inesistente”		
ID Psicologo	“”		
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione BadRequestException messaggio "Codice Fiscale Psicologo obbligatorio".			

Test case ID	TC_RF7_4	Test frame	VPZT3, VAUTH2
Pre-condizione			
Il Service è inizializzato. Paziente assegnato a un altro psicologo.			
Flusso di eventi			
1. Viene invocato validazione(). 2. Controllo associazione fallisce.			
Input	**Valore**		
ID Paziente	“paziente-inesistente”		
ID Psicologo	“CF_ALTRO”		
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione ForbiddenException con messaggio "Non sei autorizzato a terminare la cura".			

Test case ID	TC_RF7_5	Test frame	VPZT3, VAUTH3, VSTATO1
Pre-condizione			
Il Service è inizializzato. Paziente assegnato correttamente ma già terminato (stato false).			
Flusso di eventi			
1. Viene invocato validazione(). 2. Controllo stato fallisce.			
Input	**Valore**		
ID Paziente	“paziente-esistente”		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione BadRequestException con messaggio "La cura del paziente ... è già stata terminata".			

Test case ID	TC_RF7_6	Test frame	VPZT3, VAUTH3, VSTATO2
Pre-condizione			
Il Service è inizializzato. Tutti i dati sono validi e coerenti.			
Flusso di eventi			
1. Viene invocato validazione(). 2. Tutti i controlli passano.			
Input	**Valore**		
ID Paziente	“paziente-inesistente”		
ID Psicologo	“CF_VALIDO”		
Oracolo			
La validazione termina con successo.			

1.2.3.2 Unit 2: Esecuzione Terminazione Cura (Service)

Test case ID	TC_RF7_7	Test frame	DB1
Pre-condizione			
Il Service è inizializzato. Validazione superata. Errore DB simulato.			
Flusso di eventi			
1. Viene invocato terminaCura(). 2. L'operazione di UPDATE sul DB fallisce.			
Input	**Valore**		
ID Paziente	"paz-1"		
ID Psicologo	"psi-1"		
Oracolo			
L'operazione fallisce. L'eccezione del database viene propagata.			

Test case ID	TC_RF7_8	Test frame	DB2, DEL1
Pre-condizione			
Il Service è inizializzato. DB Update OK. Servizio Assegnazione simula errore.			
Flusso di eventi			
1. Viene invocato terminaCura(). 2. Update DB OK. 3. assignNextPatientToPsychologist fallisce.			
Input	**Valore**		
ID Paziente	“paz-1”		
ID Psicologo	“psi-1”		
Oracolo			
L’operazione fallisce. L’eccezione del servizio di delega viene propagata.			

Test case ID	TC_RF7_9	Test frame	DB2, DEL2
Pre-condizione			
Il Service è inizializzato. Tutto funziona correttamente.			
Flusso di eventi			
1. Viene invocato terminaCura(). 2. Update DB OK. 3. Assegnazione nuovo paziente OK.			
Input	**Valore**		
ID Paziente	“paz-1”		
ID Psicologo	“psi-1”		
Oracolo			
L’operazione termina con successo. Restituisce un oggetto con messaggio “Cura terminata con successo” e ID del nuovo paziente assegnato.			

1.2.3.3 Unit 3: Gestione Richiesta (Controller)

Test case ID	TC_RF7_10	Test frame	DLG1
Pre-condizione			
Il Controller è inizializzato. Parametri non validi.			
Flusso di eventi			
1. Viene invocato il metodo del controller.			
Input	**Valore**		
Params	{ id: “invalid-uuid” }		
Query	{ cf: “psi-1” }		
Oracolo			
Il controller (o pipe) solleva BadRequestException per validazione fallita.			

Test case ID	TC_RF7_11	Test frame	DLG2, RSP1
Pre-condizione			
Il Controller è inizializzato. Service restituisce null/undefined imprevisto.			
Flusso di eventi			
1. Il Controller è inizializzato. Service restituisce null/undefined imprevisto.			
Input	**Valore**		
Params	{ id: “invalid-uuid” }		
Query	{ cf: “psi-1” }		
Oracolo			
Il controller lancia InternalServerError.			

Test case ID	TC_RF7_12	Test frame	DLG2, RSP2
Pre-condizione			
Il Controller è inizializzato. Flusso success.			
Flusso di eventi			
1. Controller estrae parametri. 2. Chiama Service. 3. Ritorna risultato.			
Input	**Valore**		
Params	{ id: “invalid-uuid” }		
Query	{ cf: “psi-1” }		
Oracolo			
Il controller restituisce status 200 OK con il payload JSON di successo.			

1.2.4 Generazione Report (UC_PSI_5)

1.2.4.1 Unit 1: Validazione Generazione Report (Service)

Test case ID	**TC_RF5_1**	**Test frame**	**VALPZT1**
Pre-condizione			
Il Service è inizializzato. Data input non valida.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service.			
Input	**Valore**		
ID Paziente	“” (Stringa Vuota)		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L’operazione fallisce. Il metodo lancia un’eccezione NotFoundException (o database error) poiché l’ID non è valido.			

Test case ID	TC_RF5_2	Test frame	VALPZT2
Pre-condizione			
Il Service è inizializzato. Paziente non trovato nel DB.			
Flusso di eventi			
1. Viene invocato validazione(). 2. Query DB ritorna vuoto.			
Input	**Valore**		
ID Paziente	“paziente-inesistente”.		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L’operazione fallisce. Il metodo lancia un’eccezione NotFoundException con messaggio “Paziente non trovato”.			

Test case ID	TC_RF5_3	Test frame	VALPZT3
Pre-condizione			
Il Service è inizializzato. Paziente esiste.			
Flusso di eventi			
1. Viene invocato validazione().			
Input	**Valore**		
ID Paziente	“paziente-esistente”.		
ID Psicologo	“CF_VALIDO”		
Oracolo			
La validazione termina con successo.			

1.2.4.2 Unit 2: Esecuzione Generazione Report (Service)

Test case ID	**TC_RF5_4**	**Test frame**	**DATA1**
Pre-condizione			
Il Service è inizializzato. Validazione superata. Nessun dato clinico presente.			
Flusso di eventi			
1. Viene invocato generateReport(). 2. Le query su Mood, Questionari, Diario e Forum ritornano array vuoti.			
Input	**Valore**		
ID Paziente	“paziente-esistente”.		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L’operazione fallisce. Il metodo lancia NotFoundException con messaggio “Nessun dato presente per generare il report”.			

Test case ID	TC_RF5_5	Test frame	DATA2, DB1
Pre-condizione			
Il Service è inizializzato. Dati presenti. Errore DB Insert simulato.			
Flusso di eventi			
1. Viene invocato generateReport(). 2. Aggregazione dati OK. 3. Insert su tabella Report fallisce.			
Input	**Valore**		
ID Paziente	“paziente-esistente”.		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L’operazione fallisce. L’eccezione del database viene propagata.			

Test case ID	TC_RF5_6	Test frame	DATA2, DB2
Pre-condizione			
Il Service è inizializzato. Tutto funziona correttamente.			
Flusso di eventi			
1. Viene invocato generateReport(). 2. Aggregazione dati OK. 3. Insert DB OK.			
Input	**Valore**		
ID Paziente	“paziente-esistente”.		
ID Psicologo	“CF_VALIDO”		
Oracolo			
L’operazione termina con successo. Restituisce oggetto con messaggio “Report generato con successo”.			

1.2.4.3 Unit 3: Gestione Richiesta (Controller)

Test case ID	TC_RF5_7	Test frame	DLG1
Pre-condizione			
Il Controller è inizializzato. Parametri non validi (simulato).			
Flusso di eventi			
1. Viene invocato il metodo del controller.			
Input	**Valore**		
Params	{ patientId: “”}		
Query	{ cf: “CF_VALIDO” }		
Oracolo			
L’operazione fallisce (validazione service o pipe).			

Test case ID	TC_RF5_8	Test frame	DLG2, RSP1
Pre-condizione			
Il Controller è inizializzato. Service fallisce.			
Flusso di eventi			
1. Il controller invoca il service. 2. Il service lancia eccezione (es. validazione fallita).			
Input	**Valore**		
Params	{ patientId: “paziente-esistente” }		
Query	{ cf: “CF_VALIDO” }		
Oracolo			
Il controller propaga l’eccezione (es. “Service validation failed”).			

Test case ID	TC_RF5_9	Test frame	DLG2, RSP2
Pre-condizione			
Il Controller è inizializzato. Flusso success.			
Flusso di eventi			
1. Controller estrae parametri. 2. Chiama Service. 3. Ritorna risultato.			
Input	**Valore**		
Params	{ patientId: “paziente-esistente” }		
Query	{ cf: “CF_VALIDO” }		
Oracolo			
Il controller restituisce status 200/201 con payload JSON di successo.			

1.3 Test Cases Amministratore

1.3.1 Modifica Assegnazione Psicologo (UC_AS_11)

1.3.1.1 Unit 1: Validazione Assegnazione (Service)

Test case ID	**TC_RF11_1**	**Test frame**	**VPZT1**
Precondizione			
Il service è inizializzato correttamente.			
Flusso di eventi			
1. Viene invocato il metodo di validazione() del service.			
Input		Valore	
ID Paziente		“”(Stringa Vuota)	
ID Nuovo Psicologo		“CF_VALIDO”	
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione `BadRequestException` con messaggio "L'ID del paziente è obbligatorio”.			

Test case ID	TC_RF11_2	Test frame	VPZT2
Precondizione			
Il Service è inizializzato. Il Paziente specificato NON esiste nel DB.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service. 2. La query al DB restituisce 0 risultati per il paziente.			
Input		Valore	
ID Paziente		paziente-inesistente	
ID Nuovo Psicologo		CF_VALIDO	
1. L'amministratore conferma la modifica.			
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione NotFoundException con messaggio "Paziente con ID ... non trovato".			

Test case ID	TC_RF11_3	Test frame	VPZT3,VPSI1
Pre-condizione			
Il Service è inizializzato. Paziente esistente.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service.			
Input		Valore	
ID Paziente		paziente-esistente	
ID Nuovo Psicologo		“”(Stringa vuota)	
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione BadRequestException con messaggio "L'ID dello psicologo è obbligatorio."			

Test case ID	TC_RF11_4	Test frame	VPZT3,VPSI2
Precondizione			
Il Service è inizializzato. Paziente esistente. Psicologo specificato NON esiste nel DB.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service. 2. La query al DB restituisce 0 risultati per lo psicologo.			
Input		Valore	
ID Paziente		paziente-esistente	
ID Nuovo Psicologo		“CF_INESISTENTE”	
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione BadRequestException con messaggio "Psicologo con codice fiscale ... non trovato".			

Test case ID	TC_RF11_5	Test frame	VPZT3,VPSI3
Precondizione			
Il Service è inizializzato. Sia Paziente che Psicologo esistono nel DB.			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service. 2. Entrambe le query al DB restituiscono risultato.			
Input		Valore	
ID Paziente		paziente-esistente	
ID Nuovo Psicologo		“CF_VALIDO”	
Oracolo			
La validazione termina con successo e non vengono sollevate eccezioni.			

1.3.1.2 Unit 2: Esecuzione Assegnazione (Service)

Test case ID	TC_RF11_6	Test frame	DB1
Precondizione			
Il Service è inizializzato. Il Database simula un errore durante l'update.			
Flusso di eventi			
1. Viene invocato assegnaPsicologo(). 2. Il DB fallisce l'operazione di UPDATE.			
Input		Valore	
ID Paziente		“paz-1”	
ID Nuovo Psicologo		“psi-new”	
Oracolo			
L'operazione fallisce. Il metodo propaga l'eccezione del Database (o lancia una custom exception se gestita).			

Test case ID	TC_RF11_7	Test frame	DB2,DEL1
Precondizione			
Il Service è inizializzato. DB Update OK. Il servizio di delega simula un errore (es. rete, logica interna).			
Flusso di eventi			
1. Viene invocato assegnaPsicologo(). 2. Update DB OK. 3. Viene tentata la chiamata a assignNextPatientToPsychologist per il vecchio psicologo. 4. La chiamata fallisce.			
Input		Valore	
ID Paziente		“paz-1”	
ID Nuovo Psicologo		“psi-new” (Diverso da old)	
Oracolo			
L'operazione viene interrotta dall'errore del servizio di delega (propaga eccezione).			

Test case ID	TC_RF11_8	Test frame	DB2,DEL2
Precondizione			
Il Service è inizializzato. Il vecchio psicologo coincide con il nuovo.			
Flusso di eventi			
1. Viene invocato assegnaPsicologo(). 2. Update DB OK. 3. Il sistema rileva che oldPsy == newPsy. 4. La delega viene saltata.			
Input		Valore	
ID Paziente		“paz-1”	
ID Nuovo Psicologo		“psi-same” (Uguale da old)	
Oracolo			
L'operazione termina con successo. Il servizio di delega NON viene invocato.			

Test case ID	TC_RF11_9	Test frame	DB2,DEL3
Precondizione			
Il Service è inizializzato. Il vecchio psicologo è diverso dal nuovo.			
Flusso di eventi			
1. Viene invocato assegnaPsicologo(). 2. Update DB OK. 3. Il sistema rileva che oldPsy != newPsy. 4. Viene chiamato assignNextPatientToPsychologist(oldPsy).			
Input		Valore	
ID Paziente		“paz-1”	
ID Nuovo Psicologo		“psi-new” (Diverso da old)	
Oracolo			
L'operazione termina con successo. Il servizio di delega viene invocato correttamente per lo psicologo "liberato".			

1.3.1.3 Unit 3: Gestione Richiesta (Controller)

Test case ID	**TC_RF11_10**	**Test frame**	**DLG1**
Precondizione			
Il Controller è inizializzato. I parametri passati (es. UUID) non sono validi.			
Flusso di eventi			
1. Viene invocato il metodo del controller.			
Input		Valore	
Params		{id: “invalid-uuid”}	
Body		{idPsicologo: “valid-id”}	
Oracolo			
Il controller (o la pipe di validazione) solleva un'eccezione BadRequestException o il service viene chiamato con parametri inaccettabili che provocano errore.			

Test case ID	TC_RF11_11	Test frame	DLG2,RSP1
Precondizione			
Il Service ha un comportamento imprevisto (es. ritorna null quando non dovrebbe).			
Flusso di eventi			
1. Il controller invoca il service. 2. Il service non ritorna un risultato valido.			
Input		Valore	
Params		{id: “valid-uuid”}	
Body		{idPsicologo: “valid-id”}	
Oracolo			
Il controller rileva l'anomalia e lancia un'eccezione InternalServerError (o simile).			

Test case ID	TC_RF11_12	Test frame	DLG2,RSP2
Precondizione			
Tutto funziona correttamente.			
Flusso di eventi			
1. Il controller estrae i parametri. 2. Invoca il service. 3. Ritorna il risultato.			
Input		Valore	
Params		{id: “valid-uuid”}	
Body		{idPsicologo: “valid-id”}	
Oracolo			
Il controller restituisce lo status 200 OK e il messaggio di successo.			

1.3.2 Modifica Priorità Paziente (UC_AS_10)

1.3.2.1 Unit 1: Validazione Modifica Priorità (Service)

Test case ID	**TC_RF10_1**	**Test frame**	**VALP1**
Pre-condizione			
Il Service è inizializzato. Data input non valida (priorità vuota).			
Flusso di eventi			
1. Viene invocato il metodo validazione() del service.			
Input	**Valore**		
ID Paziente	"paziente-valido"		
Nuova Priorità	"" (Stringa Vuota)		
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione BadRequestException con messaggio "Priorità non valida. Valori accettati...".			

Test case ID	TC_RF10_2	Test frame	VALP2
Pre-condizione			
Il Service è inizializzato. Data input non valida (valore non accettato).			
Flusso di eventi			
1. Viene invocato validazione().			
Input	**Valore**		
ID Paziente	“paziente-valido”		
Nuova Priorità	“Inesistente”		
Oracolo			
L’operazione fallisce. Il metodo lancia un’eccezione BadRequestException con messaggio “Priorità non valida. Valori accettati…”.			

Test case ID	TC_RF10_3	Test frame	VALP3, VALPZT1
Pre-condizione			
Il Service è inizializzato. ID Paziente non valido.			
Flusso di eventi			
1. Viene invocato validazione().			
Input	**Valore**		
ID Paziente	“”		
Nuova Priorità	“Urgente”		
Oracolo			
L’operazione fallisce. Il metodo lancia un’eccezione NotFoundException (o database error) poichè la query non trova nulla con stringa vuota.			

Test case ID	TC_RF10_4	Test frame	VALP3, VALPZT2
Pre-condizione			
Il Service è inizializzato. Paziente non trovato nel DB.			
Flusso di eventi			
1. Viene invocato validazione(). 2. Query DB ritorna vuoto.			
Input	**Valore**		
ID Paziente	"paziente-inesistente"		
Nuova Priorità	"Urgente"		
Oracolo			
L'operazione fallisce. Il metodo lancia un'eccezione NotFoundException con messaggio "Paziente con ID … non trovato".			

Test case ID	TC_RF10_5	Test frame	VALP3, VALPZT3
Pre-condizione			
Il Service è inizializzato. Tutti i dati sono validi e coerenti.			
Flusso di eventi			
1. Viene invocato validazione(). 2. Tutti i controlli passano.			
Input	**Valore**		
ID Paziente	“paziente-esistente”		
Nuova Priorità	“Urgente”		
Oracolo			
La validazione termina con successo e restituisce la priorità validata.			

1.3.2.2 Unit 2: Esecuzione Modifica Priorità (Service)

Test case ID	TC_RF10_6	Test frame	DB1
Pre-condizione			
Il Service è inizializzato. Validazione (mockata) superata. Errore DB simulato.			
Flusso di eventi			
1. Viene invocato modificaPrioritaPaziente(). 2. L'operazione di UPDATE sul DB fallisce.			
Input	**Valore**		
ID Paziente	"paziente-esistente"		
Nuova Priorità	"Urgente"		
Oracolo			
L'operazione fallisce. L'eccezione del database viene propagata.			

Test case ID	TC_RF10_7	Test frame	DB2
Pre-condizione			
Il Service è inizializzato. Tutto funziona correttamente.			
Flusso di eventi			
1. Viene invocato modificaPrioritaPaziente(). 2. Update DB OK.			
Input	**Valore**		
ID Paziente	“paziente-esistente”		
Nuova Priorità	“Urgente”		
Oracolo			
L’operazione termina con successo. Restituisce oggetto con success: true e messaggio “Priorità paziente aggiornata con successo”.			

1.3.2.3 Unit 3: Gestione Richiesta (Controller)

Test case ID	TC_RF10_8	Test frame	DLG1
Pre-condizione			
Il Controller è inizializzato. Body non valido (campo mancante).			
Flusso di eventi			
1. Viene invocato il metodo modificaPriorita().			
Input	**Valore**		
ID (Param)	"paziente-esistente"		
Body	{} (Empty Object)		
Oracolo			
Il controller solleva BadRequestException con messaggio "Il campo idPriorita è obbligatorio".			

Test case ID	TC_RF10_9	Test frame	DLG2, RSP1
Pre-condizione			
Il Controller è inizializzato. Service restituisce errore.			
Flusso di eventi			
1. Il controller invoca il service. 2. Il service lancia un'eccezione (es. validazione fallita).			
Input	**Valore**		
ID (Param)	"paziente-esistente"		
Body	{ "idPriorita": "Urgente" }		
Oracolo			
L'eccezione viene propagata (es. "Service validation failed").			

Test case ID	TC_RF10_10	Test frame	DLG2, RSP2
Pre-condizione			
Il Controller è inizializzato. Flusso success.			
Flusso di eventi			
1. Controller estrae parametri. 2. Chiama Service. 3. Ritorna risultato.			
Input	**Valore**		
ID (Param)	“paziente-esistente”		
Body	{ “idPriorita”: “Urgente” }		
Oracolo			
Il controller restituisce il payload di successo fornito dal service.			

1.3.3 Accettazione Richiesta Invalidazione (UC_AS_17)

1.3.3.1 Unit 1: Validazione richiesta (Service)

Test case ID	**TC_RF17_1**	**Test frame**	**VALQ1**
Pre-condizione			
Il Service è inizializzato. ID Questionario vuoto.			
Flusso di eventi			
1. Viene invocato il metodo validazione().			
Input	**Valore**		
ID Questionario	“” (Stringa Vuota)		
Email Amministratore	“admin@test.com”		
Oracolo			
L’operazione fallisce. Eccezione: “ID Questionario obbligatorio”.			

Test case ID	TC_RF17_2	Test frame	VALQ2, VALA1
Pre-condizione			
Il Service è inizializzato. Email Amministratore vuota.			
Flusso di eventi			
1. Viene invocato il metodo validazione().			
Input	**Valore**		
ID Questionario	“valid-id”		
Email Amministratore	“” (Stringa Vuota)		
Oracolo			
L’operazione fallisce. Eccezione: “Email Amministratore obbligatoria”.			

Test case ID	TC_RF17_3	Test frame	VALQ2, VALA2, VALPQ1
Pre-condizione			
Questionario non presente nel DB.			
Flusso di eventi			
1. Viene invocato validazione(). 2. Query DB questionario ritorna null.			
Input	**Valore**		
ID Questionario	“not-found-id”		
Email Amministratore	“admin@test.com”		
Oracolo			
L’operazione fallisce. Eccezione: “Questionario non trovato”.			

Test case ID	TC_RF17_4	Test frame	VALQ2, VALA2, VALPQ2, VALPA1
Pre-condizione			
Amministratore non presente nel DB.			
Flusso di eventi			
1. Viene invocato validazione(). 2. Query DB amministratore ritorna null.			
Input	**Valore**		
ID Questionario	"valid-id"		
Email Amministratore	"unknown@test.com"		
Oracolo			
L'operazione fallisce. Eccezione: "Amministratore non trovato".			

Test case ID	TC_RF17_5	Test frame	VALQ2, VALA2, VALPQ2, VALPA2, VINV1
Pre-condizione			
Questionario già invalidato nel DB (invalidato: true).			
Flusso di eventi			
1. Viene invocato validazione(). 2. Check stato invalidazione.			
Input	**Valore**		
ID Questionario	"valid-id"		
Email Amministratore	"admin@test.com"		
Oracolo			
L'operazione fallisce. Eccezione: "Il questionario è già stato invalidato".			

Test case ID	TC_RF17_6	Test frame	VALQ2, VALA2, VALPQ2, VALPA2, VINV2
Pre-condizione			
Dati validi e presenti. Questionario non invalidato.			
Flusso di eventi			
1. Viene invocato validazione(). 2. Tutti i check passano.			
Input	**Valore**		
ID Questionario	"valid-id"		
Email Amministratore	"admin@test.com"		
Oracolo			
L'operazione termina con successo. Ritorna oggetto questionario.			

1.3.3.2 Unit 2: Ricerca precedente questionario con cambiamento (service)

Test case ID	TC_RF17_7	Test frame	VALPZT1
Pre-condizione			
ID Paziente mancante			
Flusso di eventi			
1. Invocazione metodo interno ricerca precedente.			
Input	**Valore**		
ID Paziente	“”		
Data	“2023-01-01”		
Oracolo			
Eccezione: “ID Paziente obbligatorio”.			

Test case ID	TC_RF17_8	Test frame	VALPZT2, VALD1
Pre-condizione			
Data mancante.			
Flusso di eventi			
1. Invocazione metodo interno.			
Input		**Valore**	
ID Paziente		“paziente-1”	
Data		null	
Oracolo			
Eccezione: “Data Invalidazione obbligatoria”.			

Test case ID	TC_RF17_9	Test frame	VALPZT2, VALD2
Pre-condizione			
Data nel futuro.			
Flusso di eventi			
1. Data nel futuro.			
Input		**Valore**	
ID Paziente		“paziente-1”	
Data		“FUTURE_DATE”	
Oracolo			
Eccezione: “Data futura non consentita”.			

Test case ID	TC_RF17_10	Test frame	VALPZT2, VALD3, VALPP1
Pre-condizione			
Paziente non esistente nel DB.			
Flusso di eventi			
1. Invocazione metodo. 2. Check esistenza paziente fallisce.			
Input		**Valore**	
ID Paziente		“unknown”	
Data		“2023-01-01”	
Oracolo			
Eccezione: “Paziente con ID unknown non trovato”.			

Test case ID	TC_RF17_11	Test frame	VALPZT2, VALD3, VALPP2, FIND1
Pre-condizione			
Nessun questionario precedente con cambiamento=true			
Flusso di eventi			
1. Query su questionari precedenti. 2. Nessun risultato.			
Input		**Valore**	
ID Paziente		“paziente-1”	
Data		“2023-01-01”	
Oracolo			
Ritorna null.			

Test case ID	TC_RF17_12	Test frame	VALPZT2, VALD3, VALPP2, FIND2
Pre-condizione			
Esiste questionario precedente con cambiamento=true			
Flusso di eventi			
1. Query DB trova record.			
Input		**Valore**	
ID Paziente		“paziente-1”	
Data		“2023-01-01”	
Oracolo			
Ritorna oggetto { id, data }			

1.3.3.3 Unit 3: Ricalcolo score (service)

Test case ID	TC_RF17_13	Test frame	VALPZT2, VALD3, VALPP2, FIND2
Pre-condizione			
ID Paziente vuoto.			
Flusso di eventi			
1. Viene invocato il metodo ricalcolaScore().			
Input		**Valore**	
ID Paziente		“”	
Data		“2023-01-01”	
Oracolo			
L’ID del paziente non è inserito.			

Test case ID	TC_RF17_14	Test frame	VALPZT2, VALD3, VALPP2, FIND2
Pre-condizione			
Data obbligatoria.			
Flusso di eventi			
1. Viene invocato il metodo ricalcolaScore().			
Input		**Valore**	
ID Paziente		"paz"	
Data		null	
Oracolo			
Eccezione: "Data Invalidazione obbligatoria".			

Test case ID	TC_RF17_15	Test frame	VALPZT2, VALD2
Pre-condizione			
Data nel futuro.			
Flusso di eventi			
2. Data nel futuro.			
Input		**Valore**	
ID Paziente		“paziente-1”	
Data		“FUTURE_DATE”	
Oracolo			
Eccezione: “Data futura non consentita”.			

Test case ID	TC_RF17_16	Test frame	VALPZT2, VALD3, VALPP1
Pre-condizione			
Paziente non esistente nel DB.			
Flusso di eventi			
1. Invocazione metodo. 2. Check esistenza paziente fallisce.			
Input		**Valore**	
ID Paziente		“unknown”	
Data		“2023-01-01”	
Oracolo			
Eccezione: “Paziente con ID unknown non trovato”.			

Test case ID	TC_RF17_17	Test frame	TREND_DOWN
Pre-condizione			
Fascia Minima 80. Score Pre: 82, Score Post: 70.			
Flusso di eventi			
1. Ciclo su questionari futuri. 2. Calcolo punteggi. 3. Verifica soglia.			
Input		**Valore**	
ID Paziente		“paziente-1”	
Oracolo			
ricalcolaScoreIncrementale ritorna false (Fascia persa). Nessun update cambiamento=true.			

Test case ID	TC_RF17_18	Test frame	TREND_UP
Pre-condizione			
Fascia Minima 80. Score Pre: 75, Score Post: 85.			
Flusso di eventi			
1. Verifica soglia: superata. 2. Verifica Pre: sotto soglia.			
Input		**Valore**	
ID Paziente		“paziente-1”	
Oracolo			
ricalcolaScoreIncrementale ritorna true. Update DB: cambiamento=true .			

Test case ID	TC_RF17_19	Test frame	TREND_STABLE
Pre-condizione			
Fascia Minima 80. Score Pre: 82, Score Post: 85.			
Flusso di eventi			
1. Verifica soglia: superata. 2. Verifica Pre: già sopra.			
Input		**Valore**	
ID Paziente		“paziente-1”	
Oracolo			
ricalcolaScoreIncrementale ritorna true. Update DB: cambiamento=false (Reset).			

1.3.3.4 Unit 4: Downgrade priorità (service)

Test case ID	TC_RF17_20	Test frame	VALPZT2, VALD3, VALPP2, FIND2
Pre-condizione			
ID Paziente vuoto.			
Flusso di eventi			
2. Viene invocato il metodo ricalcolaScore().			
Input		**Valore**	
ID Paziente		“”	
Data		“2023-01-01”	
Oracolo			
L’ID del paziente non è inserito.			

Test case ID	TC_RF17_21	Test frame	VALPZT2, VALD3, VALPP1
Pre-condizione			
Paziente non esistente nel DB.			
Flusso di eventi			
1. Invocazione metodo. 2. Check esistenza paziente fallisce.			
Input		**Valore**	
ID Paziente		“unknown”	
Data		“2023-01-01”	
Oracolo			
Eccezione: “Paziente con ID unknown non trovato”.			

Test case ID	TC_RF17_22	Test frame	ASP1
Pre-condizione			
Priorità attuale: “Urgente”.			
Flusso di eventi			
1. Lookup priorità paziente. 2. Mapping downgrade.			
Input		**Valore**	
ID Paziente		“paziente-urgente”	
Oracolo			
Update DB: Nuova priorità “Breve”.			

Test case ID	TC_RF17_23	Test frame	ASP1
Pre-condizione			
Priorità attuale: “Breve”.			
Flusso di eventi			
1. Lookup priorità paziente. 2. Mapping downgrade			
Input		**Valore**	
ID Paziente		“paziente-breve”	
Oracolo			
Update DB: Nuova priorità “Differibile”.			

Test case ID	TC_RF17_24	Test frame	ASP1
Pre-condizione			
Priorità attuale: “Differibile”.			
Flusso di eventi			
1. Lookup priorità paziente. 2. Mapping downgrade.			
Input		**Valore**	
ID Paziente		“paziente-differibile”	
Oracolo			
Update DB: Nuova priorità “Programmabile”.			

<table>
<tr><th>Test case ID</th><th>TC_RF17_25</th><th>Test frame</th><th>ASP1</th></tr>
<tr><th colspan="4">Pre-condizione</th></tr>
<tr><td colspan="4">Priorità attuale: “Programmabile”.</td></tr>
<tr><th colspan="4">Flusso di eventi</th></tr>
<tr><td colspan="4">3. Lookup priorità paziente.
4. Mapping downgrade.</td></tr>
<tr><th colspan="2">Input</th><th colspan="2">Valore</th></tr>
<tr><td colspan="2">ID Paziente</td><td colspan="2">“paziente-programmabile”</td></tr>
<tr><th colspan="4">Oracolo</th></tr>
<tr><td colspan="4">Update DB: Nuova priorità “Programmabile”.</td></tr>
</table>

1.3.3.5 Unit 5: Esecuzione Invalidazione (service)

Test case ID	TC_RF17_26	Test frame	DB1
Pre-condizione			
Errore simulato su DB update.			
Flusso di eventi			
1. validazione() OK. 2. Tentativo update invalidato=true.			
Input		**Valore**	
ID Quest		"valid"	
Oracolo			
Eccezione propagata. Transazione fallita.			

Test case ID	TC_RF17_27	Test frame	DB2
Pre-condizione			
Tutto ok.			
Flusso di eventi			
1. Validazione. 2. Update DB. 3. Orchestrazione (Recalc o Downgrade). 4. Notifica.			
Input		**Valore**	
ID Quest		“valid”	
Oracolo			
Nessuna eccezione. Il questionario risulta invalidato nel DB.			

1.3.3.6 Unit 6: Gestione Richiesta (Controller)

Test case ID	TC_RF17_28	Test frame	DLG1
Pre-condizione			
Parametri invalidi (UUID errato)			
Input		**Valore**	
ID		“invalid-uuid”	
Oracolo			
Status 400 Bad Request.			

Test case ID	TC_RF17_29	Test frame	DLG2, RSP1
Pre-condizione			
Service lancia errore inaspettato.			
Input		**Valore**	
ID		“invalid-uuid”	
Oracolo			
Status 500 Internal Server Error.			

Test case ID	TC_RF17_30	Test frame	DLG2, RSP2
Pre-condizione			
Esecuzione service OK.			
Input		**Valore**	
ID		“invalid-uuid”	
Oracolo			
Status 200 OK. Payload JSON successo.			

Test case ID	TC_RF17_31	Test frame	DLG2, RSP2
Pre-condizione			
Service restituisce dati.			
Input		(Request valida)	
Oracolo			
200 OK, Body JSON corretto.			